WEEK 1 & WEEK 2

Databricks Data Intelligence Platform

In-depth Study Guide for

Databricks Certified Data Engineer Associate

- Lakehouse Architecture
- Delta Lake: ACID, Time Travel, Schema
- Unity Catalog: Metastore & Namespaces
- Compute Services Deep Dive

Table of Contents

WEEK 1: DATABRICKS INTELLIGENCE PLATFORM

1. Lakehouse Architecture & Big Picture 4
2. Delta Lake Fundamentals 6
3. Delta Lake: DML & Advanced Features 9
4. Databricks Workspace & Runtime 13
5. Week 1 Practice Questions 16

WEEK 2: UNITY CATALOG & COMPUTE SERVICES

6. Unity Catalog Architecture 18
7. Managed vs External Tables 21
8. Compute Services: Clusters 24
9. SQL Warehouses & Serverless Compute 28
10. Cost Models & Workload Selection 31
11. Week 2 Practice Questions 34
12. Combined Cheat Sheet 37

WEEK 1

Week 1: Databricks Intelligence Platform

Lakehouse · Delta Lake · Workspace · Runtime

1. Lakehouse Architecture & Big Picture

The **Lakehouse** is a unified data platform architecture that combines the low-cost, scalable storage of a **data lake** with the data management and performance capabilities of a **data warehouse**. Databricks implements this through the **Data Intelligence Platform**, powered by Delta Lake and Unity Catalog.

Layer	Traditional Data Lake	Data Warehouse	Lakehouse (Databricks)
Storage	Object storage (cheap)	Proprietary (expensive)	Object storage (cheap)
Transactions	No ACID	ACID	ACID via Delta Lake
Schema	Schema-on-read	Schema-on-write	Both enforced + evolved
Data Types	Unstructured/semi-struct	Structured only	All types
BI & ML	ML friendly, poor BI	Great BI, poor ML	Both: BI + ML + streaming
Openness	Open formats	Vendor lock-in	Open: Delta, Parquet, Iceberg
Governance	Weak	Strong	Unity Catalog (strong)

1.1 Databricks Data Intelligence Platform Components

Component	Role	Key Tech
Delta Lake	Open storage layer with ACID, versioning, schema	Parquet + Delta Log (_delta_log/)
Unity Catalog	Unified governance: data + AI assets across workspaces	3-level namespace: catalog.schema.table
Apache Spark	Distributed compute engine for ETL, SQL, ML, streaming	DataFrames, Spark SQL, Structured Streaming
Lakeflow Pipelines	Declarative ETL pipelines (formerly Delta Live Tables)	LIVE tables, streaming tables, expectations
Lakeflow Jobs	Workflow orchestration: schedule & run tasks	DAG tasks, triggers, retries
MLflow	ML lifecycle: tracking, registry, deployment	Experiments, models, serving
Vector Search	AI similarity search for embeddings	Delta Sync, DLT pipelines
Databricks SQL	BI query engine on the Lakehouse	SQL Warehouses, Dashboards

■ **Exam Focus**
The exam focuses on the **Data Intelligence Platform** as a whole — understand how Delta Lake + Unity Catalog + Compute Services work together. Domain 1 is only 6% but provides the conceptual foundation for every other domain.

2. Delta Lake Fundamentals

Delta Lake is an open-source storage layer that brings reliability to data lakes. It stores data as **Parquet files** plus a **transaction log** (_delta_log/), enabling ACID properties on top of cloud object storage.

2.1 ACID Properties in Delta Lake

Property	Meaning	How Delta Implements It
Atomicity	All-or-nothing writes — partial writes never visible	Writes staged to temp files; committed atomically in _delta_log
Consistency	Data always in a valid state after any transaction	Schema enforcement rejects incompatible data before commit
Isolation	Concurrent transactions do not interfere	Optimistic concurrency control; conflict detection at commit time
Durability	Committed data persists even after failures	Transaction log + Parquet files on durable object storage (S3/ADLS/GCS)

2.2 Transaction Log (_delta_log)

Every Delta table has a **_delta_log/** directory containing numbered JSON files. Each JSON file is a **commit** that records exactly what changed (files added/removed, schema changes, metadata). This log is the source of truth for the table's state.

■ Delta Log Structure

```
# Delta table directory structure
/delta/orders/
    _delta_log/
        00000000000000000000.json   # initial commit (CREATE TABLE)
        00000000000000000001.json   # second commit (INSERT)
        00000000000000000002.json   # third commit (UPDATE)
        00000000000000000010.checkpoint.parquet  # checkpoint every 10
    part-00000-abc123.parquet
    part-00001-def456.parquet

# Every commit JSON contains:
# - commitInfo: operation, timestamp, user, parameters
# - add: files added (path, size, stats, partition values)
# - remove: files logically deleted (tombstoned)
# - metaData: schema, partition columns, config
# - protocol: min reader/writer version required
```

2.3 Time Travel

Because Delta Lake retains the transaction log, you can query any previous version of a table. This is called **Time Travel**.

■ Time Travel — All Patterns

```
# ■■■ Query by version number ■■■
df_v0 = spark.read.format('delta') \
    .option('versionAsOf', 0) \
    .load('/delta/orders')

# ■■■ Query by timestamp ■■■
```

```
df_ts = spark.read.format('delta') \
    .option('timestampAsOf', '2026-01-15 00:00:00') \
    .load('/delta/orders')

# ■■■ SQL syntax ■■■
spark.sql("SELECT * FROM orders VERSION AS OF 5")
spark.sql("SELECT * FROM orders TIMESTAMP AS OF '2026-01-15'")

# ■■■ View full history ■■■
from delta.tables import DeltaTable
dt = DeltaTable.forPath(spark, '/delta/orders')
dt.history().show(truncate=False)

# ■■■ Restore to a previous version ■■■
dt.restoreToVersion(3)              # restore to version 3
dt.restoreToTimestamp('2026-01-10') # restore to timestamp
```

Time Travel Use Case	Method	Notes
Audit/compliance query	VERSION AS OF / TIMESTAMP AS OF	Read-only; returns snapshot
Rollback bad ingestion	DeltaTable.restoreToVersion()	Creates new commit; table points to old version
Reproducible ML training	Snapshot a specific version	Pinned version never changes
Compare data over time	Query two versions and diff	JOIN two version queries on key
Undo accidental DELETE	restoreToVersion(n-1)	Must be within retention period

■■ Warning

Time Travel Retention: Delta retains history for 30 days by default (delta.deletedFileRetentionDuration = 30 days). VACUUM removes files older than the retention period — after VACUUM you cannot time-travel beyond that point. Always set retention before running VACUUM.

3. Delta Lake: DML & Advanced Features

3.1 Full DML Support

Unlike traditional data lakes (Parquet/CSV), Delta Lake supports SQL DML operations natively. These are fully ACID-compliant.

■ Delta DML — INSERT / UPDATE / DELETE / MERGE

```
# ■■■ INSERT ■■■
INSERT INTO orders VALUES (1001, 'Alice', 250.00, '2026-01-20')
INSERT INTO orders SELECT * FROM new_orders WHERE date = '2026-01-20'

# ■■■ UPDATE ■■■
UPDATE orders SET status = 'shipped' WHERE order_date < '2026-01-15'

# ■■■ DELETE ■■■
DELETE FROM orders WHERE status = 'cancelled'

# ■■■ MERGE (Upsert — most important for exam) ■■■
MERGE INTO orders AS target
USING updates AS source
ON target.order_id = source.order_id
WHEN MATCHED AND source.status = 'cancelled' THEN DELETE
WHEN MATCHED THEN UPDATE SET
    target.status = source.status,
    target.updated_at = source.updated_at
WHEN NOT MATCHED THEN INSERT *

# ■■■ PySpark MERGE ■■■
from delta.tables import DeltaTable
dt = DeltaTable.forName(spark, 'orders')
dt.alias('target').merge(
    updates_df.alias('source'),
    'target.order_id = source.order_id'
).whenMatchedUpdateAll() \
 .whenNotMatchedInsertAll() \
 .whenNotMatchedBySourceDelete() \
 .execute()
```

3.2 OPTIMIZE & Z-Order

Over time, Delta tables accumulate many small Parquet files (the 'small files problem'). **OPTIMIZE** compacts them. **Z-ORDER BY** co-locates related data within files for faster filter queries.

■ OPTIMIZE, Z-ORDER, VACUUM

```
-- Compact small files
OPTIMIZE orders

-- Compact + co-locate data by frequently filtered columns
OPTIMIZE orders ZORDER BY (customer_id, order_date)

-- PySpark equivalent
from delta.tables import DeltaTable
DeltaTable.forName(spark, 'orders').optimize().executeZOrderBy('customer_id','order_date')

-- VACUUM: remove files outside retention window
```

```
VACUUM orders RETAIN 168 HOURS   -- 7 days
VACUUM orders                    -- uses default 30 days

-- WARNING: dry run first to see what would be deleted
VACUUM orders RETAIN 168 HOURS DRY RUN
```

3.3 Schema Enforcement & Evolution

Feature	Default?	What It Does	How to Enable
Schema Enforcement	ON (always)	Rejects writes with incompatible schema	Cannot be disabled — Delta always enforces
Schema Evolution (append)	OFF	Auto-adds new columns from incoming data	.option("mergeSchema","true") or spark.databricks.delta.schema.autoMerge.enabled=true
Type widening	OFF	Allows safe type changes (int→long, float→double)	ALTER TABLE ... ALTER COLUMN ... TYPE ... or table property
Column mapping	ON (Delta 2.0+)	Rename/drop columns without rewriting data	ALTER TABLE ... RENAME COLUMN / DROP COLUMN

■ Schema Evolution & Table Properties

```
# ■■■ Schema Evolution on write ■■■
new_df.write.format('delta') \
    .option('mergeSchema', 'true') \
    .mode('append') \
    .save('/delta/orders')

# ■■■ ALTER TABLE for structural changes ■■■
ALTER TABLE orders ADD COLUMNS (discount DOUBLE AFTER amount)
ALTER TABLE orders RENAME COLUMN cust_id TO customer_id
ALTER TABLE orders DROP COLUMN internal_flag
ALTER TABLE orders ALTER COLUMN price TYPE DOUBLE

# ■■■ Table properties for Delta behavior ■■■
ALTER TABLE orders SET TBLPROPERTIES (
    'delta.deletedFileRetentionDuration' = 'interval 30 days',
    'delta.logRetentionDuration'         = 'interval 30 days',
    'delta.autoOptimize.optimizeWrite'   = 'true',
    'delta.autoOptimize.autoCompact'     = 'true'
)
```

3.4 Liquid Clustering (New in Delta 3.x)

Liquid Clustering replaces static partitioning and Z-Ordering with a dynamic, incremental approach. It reorganizes data as part of OPTIMIZE without requiring full table rewrites.

■ Liquid Clustering

```
-- Create table with Liquid Clustering
CREATE TABLE orders
CLUSTER BY (customer_id, order_date)
AS SELECT * FROM raw_orders

-- Add clustering to existing table
ALTER TABLE orders CLUSTER BY (customer_id, order_date)
```

```
-- Trigger incremental clustering
OPTIMIZE orders

-- Check clustering information
DESCRIBE DETAIL orders
```

Feature	Z-ORDER	Liquid Clustering
Applies to	Specific OPTIMIZE run	Incremental — any OPTIMIZE
Changing columns	Full rewrite needed	Just ALTER TABLE
Partition dependency	Works alongside partitions	No partitioning needed
Performance	Good	Better for high-cardinality columns
Exam relevance	Still tested	Tested as "new optimization feature"

4. Databricks Workspace & Runtime

The Databricks Workspace is the web-based collaborative environment where users write notebooks, run jobs, manage data, and configure clusters. Understanding its structure is foundational.

4.1 Workspace Components

Component	Purpose	Key Points
Notebooks	Interactive development environment	Supports Python, SQL, Scala, R; real-time collaboration; revision history
Repos / Git Folders	Version control integration	Connect to GitHub/GitLab/Azure DevOps; branch, commit, PR from UI
Workflows (Jobs)	Orchestrate multi-task pipelines	Schedule notebooks, SQL, DLT, ML; DAG dependencies
Delta Live Tables	Declarative ETL pipeline framework	LIVE TABLE, STREAMING TABLE, expectations; managed by Databricks
SQL Editor	Write and run SQL queries	Connected to SQL Warehouses; supports Dashboards
Unity Catalog UI	Browse data: catalogs, schemas, tables	View lineage, permissions, table details
Machine Learning	ML experiment tracking via MLflow	Track parameters, metrics, artifacts; model registry
Clusters UI	Manage compute resources	Create/edit/terminate; view metrics and logs
Volumes	Non-tabular file storage in Unity Catalog	Managed/External; replaces DBFS for new workloads

4.2 Databricks Runtime (DBR)

The **Databricks Runtime** is the pre-built environment that runs on clusters. It includes Apache Spark + Databricks optimizations + pre-installed libraries.

Runtime Type	Includes	Best For
Databricks Runtime (DBR)	Spark + Delta Lake + Databricks optimizations + Python/R/Scala	General ETL, data engineering, data science
DBR ML	DBR + MLflow + deep learning libraries (TensorFlow, PyTorch, scikit-learn)	Machine learning, model training
DBR Genomics	DBR + genomics libraries (Hail, GATK)	Genomics / life science workloads
Photon Runtime	DBR + Photon (C++ vectorized query engine)	High-performance SQL / BI queries; significant speedup for SQL workloads
DBR LTS	Long Term Support versions (e.g., 14.3 LTS)	Production stability; supported for 2+ years

■ **Tip**
Photon is Databricks' C++ query engine that accelerates SQL and DataFrame workloads by up to 12× on read-heavy analytical queries. It is compatible with Spark APIs but executes through a native vectorized engine. Enabled on SQL Warehouses by default.

4.3 DBFS vs Unity Catalog Volumes

Aspect	DBFS (Legacy)	Unity Catalog Volumes (Recommended)
What it is	Databricks File System — workspace-scoped virtual filesystem	Governed file storage inside Unity Catalog namespace
Governance	No Unity Catalog governance; workspace-only access	Full Unity Catalog RBAC; audited; cross-workspace
Path format	dbfs:/path/to/file	dbfs:/Volumes/catalog/schema/volume/path OR /Volumes/...
Access control	Workspace ACLs only	GRANT/REVOKE on volume like a table
Use for	Legacy workloads, init scripts	New workloads: raw files, ML artifacts, documents
Migration	Migrate to Volumes for new projects	External Volumes point to existing cloud paths

■ Unity Catalog Volumes

```
-- Create a Managed Volume (Databricks manages storage location)
CREATE VOLUME catalog_name.schema_name.my_volume

-- Create an External Volume (points to existing cloud path)
CREATE EXTERNAL VOLUME catalog_name.schema_name.ext_volume
LOCATION 's3://my-bucket/data/raw/'

-- Access files in a Volume
df = spark.read.csv('/Volumes/main/raw/landing/orders.csv', header=True)

-- Grant access
GRANT READ VOLUME ON VOLUME main.raw.landing TO `data_team`
GRANT WRITE VOLUME ON VOLUME main.raw.landing TO `ingestion_svc`
```

5. Week 1 Practice Questions

Q1. A data engineer needs to ensure that all data written to a Delta table is type-safe and conforms to its defined schema. Which Delta Lake feature enforces this automatically?

A) Time Travel B) Schema Enforcement C) MERGE INTO D) OPTIMIZE

■ **Answer:** B) Schema Enforcement — Delta Lake always enforces the schema on write. Incompatible data (wrong types, unexpected columns without mergeSchema) causes the write to fail before any data is committed.

Q2. A company must be able to query order data as it existed on January 1, 2026 for regulatory audit. Which Delta feature enables this?

A) VACUUM B) Schema Evolution C) Time Travel D) Z-ORDER

■ **Answer:** C) Time Travel — Use TIMESTAMP AS OF or VERSION AS OF to query historical snapshots. Ensure the data is within the retention window (default 30 days).

Q3. After running VACUUM RETAIN 0 HOURS on a Delta table, a data engineer tries to query VERSION AS OF 5. What happens?

A) Returns results from version 5 B) Raises an error — historical files have been deleted C) Returns current data D) Automatically restores version 5

■ **Answer:** B) Error — VACUUM deletes Parquet files outside the retention window. Without those physical files, time travel to version 5 fails. Always set retention appropriately before VACUUM.

Q4. A new data source adds an extra column 'promo_code' not present in the existing Delta table. The pipeline uses append mode. How should the engineer allow this without failing?

A) Recreate the table B) Use .option('mergeSchema','true') on the write C) Use VACUUM D) Disable schema enforcement

■ **Answer:** B) mergeSchema='true' — This triggers schema evolution, automatically adding 'promo_code' as a new column in the Delta table. Existing rows get NULL for this column.

Q5. What is the primary advantage of Liquid Clustering over Z-ORDER BY in Delta Lake?

A) Liquid Clustering is faster for point lookups B) Liquid Clustering reorganizes data incrementally without full rewrites and allows changing cluster columns via ALTER TABLE C) Liquid Clustering works only on string columns D) Liquid Clustering replaces OPTIMIZE entirely

■ **Answer:** B) Incremental + flexible — Liquid Clustering integrates with regular OPTIMIZE runs and allows changing clustering columns without rewriting the full table. Z-ORDER requires full rewrites when changing columns.

Q6. Which Databricks Runtime is pre-optimized for SQL and BI workloads with a C++ vectorized query engine?

A) DBR ML B) DBR LTS C) Photon Runtime D) DBR Genomics

■ **Answer:** C) Photon Runtime — Photon is Databricks' C++ native vectorized engine that significantly accelerates SQL-heavy workloads. SQL Warehouses use Photon by default.

Q7. A data team needs to store raw JSON files under Unity Catalog governance with full access control and auditability. What is the recommended storage object?

A) DBFS mount point B) Unity Catalog Volume C) Delta table D) Workspace file system

■ **Answer:** B) Unity Catalog Volume — Volumes provide governed, audited file storage inside the Unity Catalog namespace. Use GRANT READ/WRITE VOLUME for access control.

Q8. What is the key architectural difference between a data lake and a Lakehouse?

A) Lakehouses use proprietary storage formats B) Lakehouses add ACID transactions, schema enforcement, and governance to a data lake C) Data lakes support BI tools natively D) Lakehouses do not support ML workloads

■ **Answer:** B) ACID + governance on open storage — The Lakehouse keeps the cheap, scalable object storage of a lake but adds the reliability of a warehouse (ACID, schema, BI performance) via Delta Lake and Unity Catalog.

WEEK 2

Week 2: Unity Catalog & Compute Services

UC Architecture · Clusters · SQL Warehouses · Cost

6. Unity Catalog Architecture

Unity Catalog (UC) is Databricks' unified governance solution for all data and AI assets. It provides a single place to manage access, audit, lineage, and discovery across multiple workspaces and clouds.

6.1 Three-Level Namespace

Unity Catalog organizes assets in a strict three-level hierarchy. All table references must use this format: **catalog.schema.table**

■ Unity Catalog — Namespace and Object Creation

```
-- Three-level namespace: catalog.schema.table_or_object

-- Examples
SELECT * FROM prod_catalog.sales.orders
SELECT * FROM dev_catalog.staging.raw_events

-- Set default catalog and schema (avoids repeating prefix)
USE CATALOG prod_catalog
USE SCHEMA sales
SELECT * FROM orders   -- now resolves to prod_catalog.sales.orders

-- Create objects in the hierarchy
CREATE CATALOG IF NOT EXISTS prod_catalog
    COMMENT 'Production data catalog';

CREATE SCHEMA IF NOT EXISTS prod_catalog.sales
    COMMENT 'Sales domain data'
    MANAGED LOCATION 's3://bucket/prod/sales/';

CREATE TABLE prod_catalog.sales.orders (
    order_id    BIGINT NOT NULL,
    customer_id BIGINT,
    amount      DECIMAL(10,2),
    order_date  DATE
) USING DELTA;
```

Level	Object	Scope	Managed By
Level 0	Metastore	1 per region; attached to workspaces	Account Admin
Level 1	Catalog	Top-level container; maps to a domain/env	Catalog Owner / Account Admin
Level 2	Schema (Database)	Groups related tables/views/functions/volumes	Schema Owner
Level 3	Table / View / Volume / Function / Model	The actual data asset	Object Owner / Granted user

6.2 Securable Objects & Privilege Hierarchy

Securable Object	Grantable Privileges	Inherits Down?
Metastore	CREATE CATALOG, CREATE EXTERNAL LOCATION, CREATE STORAGE CREDENTIAL	Yes → catalogs
Catalog	CREATE SCHEMA, USE CATALOG, ALL PRIVILEGES	Yes → schemas
Schema	CREATE TABLE, CREATE VIEW, CREATE VOLUME, USE SCHEMA	Yes → tables/views
Table	SELECT, MODIFY, ALL PRIVILEGES	—
View	SELECT	—
Volume	READ VOLUME, WRITE VOLUME	—
Function	EXECUTE	—

■ GRANT / REVOKE / DENY Syntax

```
-- Grant privileges
GRANT USE CATALOG ON CATALOG prod_catalog TO `analysts_group`
GRANT USE SCHEMA  ON SCHEMA prod_catalog.sales TO `analysts_group`
GRANT SELECT      ON TABLE prod_catalog.sales.orders TO `analysts_group`

-- Grant on all current tables in a schema
GRANT SELECT ON ALL TABLES IN SCHEMA prod_catalog.sales TO `analysts_group`

-- Revoke
REVOKE SELECT ON TABLE prod_catalog.sales.orders FROM `contractors`

-- Show grants
SHOW GRANTS ON TABLE prod_catalog.sales.orders
SHOW GRANTS TO `analysts_group`

-- Deny (explicit deny overrides grants from groups)
DENY SELECT ON TABLE prod_catalog.sales.pii_data TO `external_user`
```

> ■ **Exam Focus**
> ■ **Privilege inheritance:** Granting USE CATALOG does NOT automatically grant USE SCHEMA or SELECT. Each level requires explicit grants. However, granting ALL PRIVILEGES on a catalog does propagate down. **DENY always overrides GRANT** — if a user is in two groups where one has GRANT and one has DENY, DENY wins.

7. Managed vs External Tables

One of the most heavily tested topics in Domain 7 (Governance). The distinction affects where data lives and what happens on DROP.

Aspect	Managed Table	External Table
Data location	Databricks/Unity Catalog manages storage location (in metastore managed location)	User specifies LOCATION — data lives in cloud storage you control
DROP TABLE behavior	Drops metadata AND deletes underlying data files	Drops metadata ONLY — data files remain in cloud storage
CLONE behavior	Data copied to managed location	Data stays at original location; metadata cloned
Format	Always Delta in Unity Catalog	Can be Delta, Parquet, CSV, JSON, ORC, Avro
Access pattern	Recommended for new Lakehouse tables	Use when data must remain in external storage (compliance, existing data)
Created with	CREATE TABLE ... AS SELECT ... (no LOCATION)	CREATE TABLE ... LOCATION 's3://...'
Ownership	Databricks owns the files	User/organization owns the files

■ Managed vs External Table Operations

```
-- ■■■ Create Managed Table (recommended) ■■■
CREATE TABLE prod_catalog.sales.orders_managed (
    order_id BIGINT, customer_id BIGINT, amount DOUBLE, order_date DATE
) USING DELTA;

-- or via CTAS (Create Table As Select)
CREATE TABLE prod_catalog.sales.orders_managed
AS SELECT * FROM raw_orders;

-- ■■■ Create External Table ■■■
CREATE TABLE prod_catalog.sales.orders_external
USING DELTA
LOCATION 's3://my-bucket/delta/orders/';

-- ■■■ Check table type ■■■
DESCRIBE EXTENDED prod_catalog.sales.orders_managed;
-- Look for: Type = MANAGED or EXTERNAL
-- Look for: Location = ... (shows storage path)

-- ■■■ Convert External to Managed ■■■
-- Step 1: Create new managed table
CREATE TABLE prod_catalog.sales.orders_managed
AS SELECT * FROM prod_catalog.sales.orders_external;

-- Step 2: Drop external (data stays!)
DROP TABLE prod_catalog.sales.orders_external;

-- ■■■ Convert Managed to External ■■■
-- Not directly possible; must CREATE EXTERNAL TABLE LOCATION ...
-- then INSERT INTO ... SELECT * FROM managed, then DROP managed
```

7.1 Data Lineage in Unity Catalog

Unity Catalog automatically captures column-level lineage — tracking how data flows from source tables through transformations to downstream tables and dashboards. No configuration needed.

Lineage Feature	What It Tracks	How to View
Table lineage	Upstream sources, downstream consumers	UC Explorer → Table → Lineage tab
Column lineage	Which source columns each target column derives from	UC Explorer → Column → Lineage
Notebook lineage	Which notebooks read/wrote the table	Table → Lineage → Source/Target notebooks
Dashboard lineage	Which dashboards query the table	Table → Lineage → Consumers
Job lineage	Which jobs produce/consume the table	Table → Lineage → Source/Target jobs

8. Compute Services: Clusters Deep Dive

Selecting the right compute type is critical for cost, performance, and compliance. Domain 1 tests your ability to match workload requirements to the appropriate compute.

Cluster Type	Purpose	Key Characteristics	Exam Scenario
All-Purpose Cluster	Interactive development, ad hoc analysis	Long-running; shared by team; manual start/stop; expensive per hour	Data engineers writing notebooks; collaborative exploration
Job Cluster	Running automated Databricks Jobs	Created fresh per job run; terminated when job completes; isolated; cheaper	Production ETL pipelines; scheduled workloads
Instance Pool	Pre-provisioned VMs to reduce startup time	Nodes pre-warmed; clusters draw from pool; reduces cold start by 80%	Jobs needing fast startup; spiky workloads
Single-Node Cluster	Dev/test or small data work on one machine	Driver only (no executors); no parallelism; cheap for light tasks	Notebook prototyping; small dataset exploration

8.1 Cluster Configuration

■ **Cluster Configuration Reference**

```
# Key cluster config parameters (via UI or Jobs API)

cluster_config = {
    'cluster_name': 'prod-etl-cluster',
    'spark_version': '14.3.x-scala2.12',   # DBR version
    'node_type_id': 'i3.xlarge',            # instance type
    'driver_node_type_id': 'i3.2xlarge',    # driver (optional override)

    # Autoscaling (recommended for variable workloads)
    'autoscale': {
        'min_workers': 2,
        'max_workers': 10
    },

    # OR fixed workers
    # 'num_workers': 4,

    # Auto-termination (saves cost)
    'autotermination_minutes': 30,

    # Spot instances (cheaper but interruptible)
    'aws_attributes': {
        'first_on_demand': 1,               # 1 on-demand driver
        'availability': 'SPOT_WITH_FALLBACK',
        'spot_bid_price_percent': 100
    },

    # Spark config
    'spark_conf': {
        'spark.sql.shuffle.partitions': '200',
        'spark.databricks.delta.preview.enabled': 'true'
    },
```

```
    # Libraries to install on cluster
    'libraries': [
        {'pypi': {'package': 'great-expectations==0.18.0'}},
        {'maven': {'coordinates': 'com.databricks:spark-xml_2.12:0.17.0'}}
    ]
}
```

8.2 Autoscaling Behavior

Autoscaling Aspect	Behavior	Implication
Scale up trigger	Active tasks > available slots for > 1 minute	Short bursts may not trigger scale-up
Scale down trigger	Worker idle for > scale-down interval (default ~10 min)	Workers stay running during brief idle periods
Enhanced Autoscaling (Photon)	More aggressive scale-down based on shuffle data	Better cost efficiency for SQL workloads
Spot interruption	Databricks gracefully requeues tasks from terminated spot VMs	Use SPOT_WITH_FALLBACK for production reliability
Min workers	Minimum always running (even when idle)	Set min=0 only for batch jobs; keep min>=1 for interactive

9. SQL Warehouses & Serverless Compute

9.1 SQL Warehouses

SQL Warehouses (formerly SQL Endpoints) are purpose-built compute for SQL analytics and BI workloads. They support Photon by default and are optimized for concurrent users running queries via the SQL Editor, dashboards, or BI tools (Tableau, Power BI, Looker).

SQL Warehouse Type	Description	Best For
Classic SQL Warehouse	Managed Spark clusters for SQL; Photon-enabled; auto-stops	Standard BI workloads; predictable query patterns
Pro SQL Warehouse	Classic + Serverless-like fast startup; enhanced features	Mixed interactive + ETL SQL workloads
Serverless SQL Warehouse	Instant startup (<3 sec); fully managed by Databricks; no cluster management	Ad hoc queries; unpredictable workloads; fastest iteration

9.2 Serverless Compute

Serverless compute eliminates the need to manage cluster infrastructure. Databricks provisions, scales, and terminates compute automatically. Available for both SQL Warehouses and Notebooks/Jobs.

Aspect	Classic Cluster	Serverless Compute
Startup time	2–5 minutes (cold)	< 3 seconds (instant)
Infrastructure management	User manages (instance type, DBR version, autoscaling)	Fully managed by Databricks; zero cluster config
Cost model	DBUs/hour while cluster is running (even when idle)	DBUs billed per query/job execution second — no idle cost
Network isolation	VPC injection possible; customer VPC	Databricks-managed network (Serverless Network Config for isolation)
Photon	Optional (add to cluster config)	Always included
Cold start	Significant (VM provisioning)	Near-zero (pre-provisioned pool)
Use case	Long-running ETL; custom configs	Ad hoc SQL; short notebook runs; cost-sensitive workloads

■ **Exam Focus**
■ **Exam Scenario:** 'Multiple business analysts run ad hoc SQL queries throughout the day needing fast startup and concurrent user support while managing cost.' → Answer: **Serverless SQL Warehouse** or **Pro SQL Warehouse with autoscaling**. Not a Job cluster (wrong use case) and not a single-node cluster (no concurrency).

10. Cost Models & Workload Selection

10.1 Databricks Unit (DBU) Pricing Model

Databricks charges in **DBUs** (Databricks Units) — a unit of processing capability per hour. DBU cost varies by compute type, cloud provider, and tier (Standard/Premium/Enterprise). You also pay cloud provider VM costs separately.

Compute Type	DBU Rate	Idle Cost	Key Cost Factor
All-Purpose Cluster	Highest DBU rate	Yes — runs even when idle	Long-running notebooks; collaborative work
Job Cluster	Lower DBU rate	No — terminates when job finishes	Production ETL/scheduled pipelines
SQL Warehouse (Classic)	Medium DBU rate	No — auto-stops after idle threshold	BI queries; Dashboards
Serverless SQL Warehouse	Pay per query second	No — truly zero when idle	Most cost-efficient for sporadic queries
Serverless Compute (Jobs)	Per second billing	No idle cost	Short-running automated jobs
Instance Pool	VM cost while pre-provisioned	Low — just VM cost (no DBUs)	Reduce startup for frequent Job clusters

10.2 Workload-to-Compute Decision Framework

■ **Compute Selection Decision Tree**

```
# Decision tree for compute selection:

if workload == 'interactive_development':
    if team_size > 1 and collaboration_needed:
        compute = 'All-Purpose Cluster (shared)'
    else:
        compute = 'All-Purpose Cluster (personal) or Serverless Notebook'

elif workload == 'ad_hoc_sql_bi':
    if startup_speed == 'critical' or users > 5:
        compute = 'Serverless SQL Warehouse'
    else:
        compute = 'Pro or Classic SQL Warehouse'

elif workload == 'scheduled_etl_pipeline':
    if startup_time_matters and runs_frequently:
        compute = 'Job Cluster from Instance Pool'
    else:
        compute = 'Job Cluster (fresh per run)'

elif workload == 'streaming_pipeline':
```

```
    compute = 'All-Purpose Cluster or Lakeflow Pipeline compute'

elif workload == 'ml_training':
    if large_model or gpu_needed:
        compute = 'All-Purpose Cluster with GPU instances + DBR ML'
    else:
        compute = 'Job Cluster + DBR ML'

elif workload == 'light_development_small_data':
    compute = 'Single-Node Cluster'
```

Scenario	Best Compute	Why
Team of 5 analysts running SQL dashboards all day	Serverless SQL Warehouse + autoscaling	Fast startup, concurrent users, cost only when querying
Nightly ETL job: 2TB Delta table, runs 45 min	Job Cluster (from Instance Pool)	Isolated per run; pool reduces startup; auto-terminated when done
Data scientist exploring 500MB CSV interactively	Single-Node Cluster or Serverless Notebook	No need for distributed compute; cheapest option
Real-time streaming from Kafka to Delta	All-Purpose or Lakeflow Pipeline compute	Long-running; must stay alive; streaming semantics
Monthly compliance report, complex SQL, 2 users	Pro SQL Warehouse (auto-stop 15 min)	Photon for speed; cost-effective with auto-stop for infrequent use
Prototype + production: same ETL code in dev and prod	Serverless Notebook (dev) + Job Cluster (prod)	Fast iteration in dev; isolated + cheap in prod

11. Week 2 Practice Questions

Q1. A data engineer drops a managed Delta table in Unity Catalog. What happens to the underlying data files?

A) Files remain in cloud storage B) Files are moved to trash C) Files are permanently deleted D) Files are archived

■ **Answer:** C) Deleted — For MANAGED tables, DROP TABLE removes both the metadata AND the underlying Parquet/Delta files from storage. This is why managed tables are NOT appropriate when you need to retain data independently.

Q2. A team needs to keep raw data in their own S3 bucket for compliance while still querying it via Unity Catalog. Which table type should they use?

A) Managed Table B) External Table C) Delta Live Table D) Streaming Table

■ **Answer:** B) External Table — EXTERNAL tables store metadata in Unity Catalog but leave data files in the specified LOCATION (e.g., s3://...). DROP TABLE only removes metadata; data stays in S3.

Q3. A user is a member of group 'analysts' which has SELECT on prod.sales.orders. The user is also in group 'contractors' which has DENY SELECT on the same table. Can the user query the table?

A) Yes — GRANT overrides DENY for individual grants B) No — DENY always overrides GRANT in Unity Catalog C) Yes — group membership grants take priority D) Depends on role hierarchy

■ **Answer:** B) DENY wins — In Unity Catalog, explicit DENY always overrides any GRANT, regardless of how the grant was obtained (individual or group). This is critical for fine-grained security.

Q4. Which Unity Catalog privilege level must be granted BEFORE a user can SELECT from any table in a schema?

A) SELECT on all tables B) USE CATALOG on the catalog, then USE SCHEMA on the schema C) MODIFY on the schema D) CREATE TABLE on the schema

■ **Answer:** B) USE CATALOG + USE SCHEMA — These 'use' privileges are required to navigate the namespace. Without them, even SELECT on a table fails because the user cannot 'see' the catalog or schema.

Q5. A company wants to reduce ETL job startup time from 4 minutes to under 30 seconds without switching to serverless compute. What should they configure?

A) Increase autoscaling max workers B) Use an Instance Pool to pre-provision VM instances C) Switch to a larger instance type D) Increase spark.sql.shuffle.partitions

■ **Answer:** B) Instance Pool — Pre-provisioned VMs start in seconds vs minutes because the VM is already running. The cluster just needs to install Databricks Runtime on the pre-warmed nodes.

Q6. What is the key billing difference between a Job Cluster and an All-Purpose Cluster?

A) Job Clusters use more DBUs per hour B) All-Purpose Clusters auto-terminate faster C) Job Clusters only incur costs when running a job (no idle cost); All-Purpose Clusters charge while running even when idle D) There is no billing difference

■ **Answer:** C) No idle cost for Job Clusters — Job Clusters are created fresh per job run and terminated automatically when the job finishes. All-Purpose Clusters continue charging DBUs while running, even if no code is executing.

Q7. A BI team reports that their SQL dashboard queries take 4+ minutes after the warehouse has been idle. The team wants sub-5 second startup. Which SQL Warehouse type best addresses this?

A) Classic SQL Warehouse with autoscaling B) Pro SQL Warehouse C) Serverless SQL Warehouse D) Increase cluster workers on Classic Warehouse

■ **Answer:** C) Serverless SQL Warehouse — Serverless provides near-instant startup (<3 seconds) because Databricks pre-provisions compute in shared pools. This directly solves the cold-start problem.

Q8. In Unity Catalog, a data engineer creates a table in catalog 'prod', schema 'finance'. To query it as 'transactions', a BI analyst needs which minimum set of privileges?

A) SELECT on the table only B) USE CATALOG on prod, USE SCHEMA on finance, SELECT on transactions C) ALL PRIVILEGES on prod D) CREATE TABLE on finance and SELECT on transactions

■ **Answer:** B) Three-level access — USE CATALOG (to access the catalog), USE SCHEMA (to access the schema), SELECT (to query the table). This three-step privilege model is fundamental to Unity Catalog security.

Cheat Sheet

Quick Reference — Week 1 & 2

Delta Lake Quick Reference

Concept	Key Fact
Transaction log	_delta_log/*.json — every commit recorded; source of truth
Time Travel	VERSION AS OF n / TIMESTAMP AS OF 'date' / restoreToVersion(n)
VACUUM default	Removes files > 30 days; cannot time-travel past retention after VACUUM
Schema enforcement	Always ON; incompatible writes fail before any data lands
Schema evolution	.option("mergeSchema","true") on write — adds new columns automatically
MERGE INTO	Upsert: WHEN MATCHED UPDATE / WHEN NOT MATCHED INSERT / WHEN NOT MATCHED BY SOURCE DELETE
OPTIMIZE	Compacts small files; add ZORDER BY for data skipping
Liquid Clustering	CLUSTER BY columns — incremental; changeable via ALTER TABLE without full rewrite
Delta table property	ALTER TABLE t SET TBLPROPERTIES('delta.deletedFileRetentionDuration'='interval 7 days')

Unity Catalog Quick Reference

Concept	Key Fact
Namespace	3 levels: catalog.schema.table (always use full path or SET DEFAULT)
Managed table	DROP = metadata + data deleted; data owned by Databricks
External table	DROP = metadata only; data stays in LOCATION path; user owns data
Convert external→managed	CREATE managed AS SELECT * FROM external; DROP external
Minimum SELECT access	USE CATALOG + USE SCHEMA + SELECT (all 3 required)
DENY behavior	DENY always wins over GRANT regardless of group membership
Lineage	Auto-captured at column level; view in UC Explorer → Table → Lineage
Volume	Governed file storage; GRANT READ/WRITE VOLUME; replaces DBFS for new work
ABAC	Attribute-Based Access Control for row/column filtering policies

Compute Services Quick Reference

Compute	Use Case	Cost Model	Startup
All-Purpose Cluster	Interactive dev, notebooks, streaming	DBU/hr while running (incl idle)	2–5 min (cold)
Job Cluster	Automated scheduled ETL jobs	DBU/hr only when job runs	2–5 min (or ~30s from pool)
Instance Pool	Reduce startup for Job Clusters	VM cost while pre-provisioned	~30 sec from pool

Single-Node	Dev/test, small data, ML on one node	Cheapest; no executor nodes	1–2 min
SQL Warehouse	BI queries, dashboards, concurrent SQL	DBU while running; auto-stop	2 min (Classic), <3s (Serverless)
Serverless SQL WH	Ad hoc SQL, max concurrency, fast	Per query second; zero idle	< 3 seconds
Serverless Compute	Short jobs, notebooks without cluster mgmt	Per second execution only	< 3 seconds

■ **Exam Focus**
■ **Top 5 Exam Tips for Week 1–2:**
1. **Managed DROP = data deleted**; External DROP = data stays
2. **DENY wins** over GRANT always in Unity Catalog
3. **USE CATALOG + USE SCHEMA + SELECT** all required for table access
4. **VACUUM** deletes time travel history — set retention before running
5. **Serverless = fastest startup + zero idle cost**; Job Cluster = cheapest for batch ETL

BONUS

Deep Dive: Optimization Features

Predictive Optimization · Data Skipping · Statistics

A1. Predictive Optimization

Predictive Optimization is a Databricks-managed feature that automatically runs OPTIMIZE and VACUUM on Unity Catalog tables when beneficial, based on usage patterns and table statistics — without requiring manual scheduling.

Aspect	Details
What it does	Automatically runs OPTIMIZE (with Z-ORDER or Liquid Clustering) and VACUUM based on table access patterns
Who controls it	Databricks — no manual configuration of schedules required
Availability	Unity Catalog managed tables on Premium/Enterprise tier
How to enable	Enable at catalog, schema, or table level via TBLPROPERTIES or UI
Benefit	Eliminates manual maintenance jobs; consistent file compaction; no forgetting VACUUM
Exam angle	Understand it as the "automated" alternative to manual OPTIMIZE/VACUUM jobs

■ Predictive Optimization

```
-- Enable Predictive Optimization on a table
ALTER TABLE prod_catalog.sales.orders
SET TBLPROPERTIES ('delta.predictiveOptimization' = 'enable')

-- Enable on schema (all current + future tables)
ALTER SCHEMA prod_catalog.sales
SET DBPROPERTIES ('delta.predictiveOptimization' = 'enable')

-- Check optimization history
SELECT * FROM prod_catalog.sales.orders
SHOW TBLPROPERTIES prod_catalog.sales.orders
```

A2. Data Skipping & Statistics

Delta Lake automatically collects column-level statistics (min, max, null count, distinct count) on the first 32 columns during write. These statistics enable **data skipping** — skipping Parquet files whose statistics show they cannot contain matching rows.

■ Data Skipping Statistics

```
-- Check statistics on a Delta table
DESCRIBE DETAIL orders
-- Look for: numFiles, sizeInBytes, statistics

-- Force statistics refresh
ANALYZE TABLE orders COMPUTE STATISTICS FOR ALL COLUMNS

-- Check which columns have statistics
ANALYZE TABLE orders COMPUTE STATISTICS FOR COLUMNS customer_id, order_date

-- View statistics (internal)
```

```
SELECT * FROM delta.`/delta/orders`
  -- or via SQL: DESCRIBE DETAIL shows aggregate stats

-- Statistics collected automatically on:
-- - INSERT (for new files)
-- - OPTIMIZE (recomputed for compacted files)
-- - ANALYZE TABLE (force full refresh)
```

■ **Tip**

Data Skipping vs Partitioning vs Z-ORDER:

- Partitioning: directory-level skipping (coarse; good for low-cardinality like date/region)
- Z-ORDER: file-level co-location for high-cardinality columns (customer_id, device_id)
- Data Skipping: always active via column statistics — even without partitioning or Z-ORDER
- Liquid Clustering: next-gen replacement for both partitioning and Z-ORDER

B. Advanced Unity Catalog: Masking & Row Security

Unity Catalog provides advanced data security features that restrict what data different users can see — without duplicating tables or building complex ETL.

B1. Column-Level Masking

Column masking allows you to show different users different versions of a column's value based on their identity or group membership — for example, masking credit card numbers for analysts but showing them to fraud investigators.

■ Column Masking

```
-- Step 1: Create a masking function
CREATE FUNCTION prod_catalog.security.mask_ssn(ssn STRING)
RETURN CASE
    WHEN is_account_group_member('pii_admins') THEN ssn
    ELSE CONCAT('***-**-', RIGHT(ssn, 4))
END;

-- Step 2: Apply mask to column
ALTER TABLE prod_catalog.hr.employees
ALTER COLUMN social_security_number
SET MASK prod_catalog.security.mask_ssn;

-- Users in 'pii_admins' see: '123-45-6789'
-- All other users see:       '***-**-6789'

-- Step 3: Drop mask if needed
ALTER TABLE prod_catalog.hr.employees
ALTER COLUMN social_security_number DROP MASK;
```

B2. Row-Level Security (Row Filters)

Row filters restrict which rows a user can see when querying a table — for example, sales reps can only see orders from their own region.

■ Row-Level Security (Row Filters)

```
-- Step 1: Create a row filter function
CREATE FUNCTION prod_catalog.security.filter_by_region(region STRING)
RETURN is_account_group_member('global_viewers')
    OR region = session_context('user_region');

-- Step 2: Apply row filter to table
ALTER TABLE prod_catalog.sales.orders
SET ROW FILTER prod_catalog.security.filter_by_region ON (region);

-- Users in 'global_viewers': see ALL rows
-- Other users: only see rows WHERE region = their region

-- Step 3: Remove row filter
ALTER TABLE prod_catalog.sales.orders DROP ROW FILTER;

-- NOTE: Row filters and column masks are transparent —
-- users query the table normally; filtering is automatic
```

B3. ABAC — Attribute-Based Access Control

Security Feature	Scope	Use Case	How Applied
Column Masking	Single column	PII masking, credit cards, SSN	ALTER TABLE ... ALTER COLUMN ... SET MASK function
Row Filter	Entire table rows	Multi-tenant: users see only their data	ALTER TABLE ... SET ROW FILTER function ON (col)
ABAC Policy	Centralized across tables	Enterprise-wide governance policy	Unity Catalog policy UI; applies to all tables in catalog/schema
GRANT/REVOKE	Object level	Standard access control	SQL: GRANT privilege ON object TO principal
Column-level GRANT	Specific columns	Restrict visibility of sensitive cols without masking	GRANT SELECT(col1, col2) ON TABLE t TO group

LAB

Lab Exercises

Hands-On: Delta Lake & Unity Catalog

C1. Lab — Delta Lake Full Lifecycle

■ Lab C1 — Delta Lake Full Lifecycle

```
# ■■■ Setup ■■■
spark.sql('CREATE SCHEMA IF NOT EXISTS lab.delta_lab')

# ■■■ Step 1: Create Delta table ■■■
spark.sql('''
    CREATE TABLE lab.delta_lab.orders (
        order_id    BIGINT NOT NULL,
        customer_id BIGINT,
        product     STRING,
        amount      DOUBLE,
        status      STRING,
        order_date  DATE
    ) USING DELTA
    COMMENT 'Lab orders table'
''')

# ■■■ Step 2: Insert data (Version 0 → Version 1) ■■■
spark.sql('''
    INSERT INTO lab.delta_lab.orders VALUES
    (1001, 1, 'Laptop', 1200.00, 'pending', '2026-01-10'),
    (1002, 2, 'Phone',   800.00, 'pending', '2026-01-11'),
    (1003, 3, 'Tablet',  450.00, 'shipped', '2026-01-12')
''')

# ■■■ Step 3: Update (Version 2) ■■■
spark.sql("UPDATE lab.delta_lab.orders SET status='shipped' WHERE order_id=1001")

# ■■■ Step 4: Delete (Version 3) ■■■
spark.sql("DELETE FROM lab.delta_lab.orders WHERE status='pending'")

# ■■■ Step 5: View history ■■■
spark.sql('DESCRIBE HISTORY lab.delta_lab.orders').show(truncate=False)

# ■■■ Step 6: Time Travel ■■■
print('Current (v3):'); spark.sql('SELECT * FROM lab.delta_lab.orders').show()
print('Version 1:');    spark.sql('SELECT * FROM lab.delta_lab.orders VERSION AS OF 1').show()
print('Version 2:');    spark.sql('SELECT * FROM lab.delta_lab.orders VERSION AS OF 2').show()

# ■■■ Step 7: Restore ■■■
spark.sql('RESTORE TABLE lab.delta_lab.orders TO VERSION AS OF 1')
spark.sql('SELECT * FROM lab.delta_lab.orders').show()  # back to 3 rows

# ■■■ Step 8: Schema Evolution ■■■
from pyspark.sql import Row
new_data = spark.createDataFrame([Row(order_id=1004, customer_id=4, product='Watch',
    amount=300.0, status='pending', order_date='2026-01-15', promo_code='SAVE10')])
new_data.write.format('delta').option('mergeSchema','true').mode('append')
    .saveAsTable('lab.delta_lab.orders')

spark.sql('DESCRIBE lab.delta_lab.orders').show()  # promo_code now appears
```

```
# ■■■ Step 9: OPTIMIZE + VACUUM ■■■
spark.sql('OPTIMIZE lab.delta_lab.orders')
spark.sql('VACUUM lab.delta_lab.orders RETAIN 168 HOURS DRY RUN')
```

C2. Lab — Unity Catalog Governance

■ **Lab C2 — Unity Catalog Governance**

```
-- ■■■ Setup (requires admin or catalog owner) ■■■

-- Create catalog structure
CREATE CATALOG IF NOT EXISTS lab_catalog COMMENT 'Training lab catalog';
CREATE SCHEMA IF NOT EXISTS lab_catalog.hr COMMENT 'HR schema';
CREATE SCHEMA IF NOT EXISTS lab_catalog.sales COMMENT 'Sales schema';

-- ■■■ Exercise 1: Managed vs External Table ■■■

-- Managed table (Databricks manages storage)
CREATE TABLE lab_catalog.sales.orders_managed (
    order_id BIGINT, amount DOUBLE, region STRING
) USING DELTA;

INSERT INTO lab_catalog.sales.orders_managed VALUES
(1, 500.0, 'West'), (2, 300.0, 'East'), (3, 750.0, 'West');

-- External table (data lives in specified location)
CREATE TABLE lab_catalog.sales.orders_external
USING DELTA LOCATION 's3://training-bucket/delta/orders_ext/';

-- Verify table types
DESCRIBE EXTENDED lab_catalog.sales.orders_managed;   -- Type = MANAGED
DESCRIBE EXTENDED lab_catalog.sales.orders_external;  -- Type = EXTERNAL

-- ■■■ Exercise 2: Access Control ■■■

-- Grant tiered access
GRANT USE CATALOG ON CATALOG lab_catalog TO `lab_analysts`;
GRANT USE SCHEMA ON SCHEMA lab_catalog.sales TO `lab_analysts`;
GRANT SELECT ON TABLE lab_catalog.sales.orders_managed TO `lab_analysts`;

-- Verify: as analyst user, this should work:
SELECT * FROM lab_catalog.sales.orders_managed;

-- Verify: this should FAIL (no access to hr schema):
SELECT * FROM lab_catalog.hr.employees;  -- Error: USE SCHEMA not granted

-- ■■■ Exercise 3: Column Masking ■■■
CREATE TABLE lab_catalog.hr.employees (
    emp_id INT, name STRING, salary DOUBLE, ssn STRING
) USING DELTA;

INSERT INTO lab_catalog.hr.employees VALUES
(1,'Alice',95000.0,'123-45-6789'), (2,'Bob',88000.0,'987-65-4321');

CREATE FUNCTION lab_catalog.hr.mask_salary(sal DOUBLE)
RETURN CASE WHEN is_account_group_member('hr_managers') THEN sal
            ELSE -1.0 END;
```

```
ALTER TABLE lab_catalog.hr.employees
ALTER COLUMN salary SET MASK lab_catalog.hr.mask_salary;

-- hr_managers see: 95000.0
-- Others see:      -1.0
```

Mock Exam

20 Questions — Timed Practice

D. Mock Exam — 20 Questions (Domain 1 Focus)

■■ **Warning**
■■ Set a 40-minute timer. Answer all questions before checking the answer key.

1. A data engineer wants to query a Delta table as it existed exactly 30 days ago. Which approach works?
A) SELECT * FROM orders OFFSET 30 DAYS B) SELECT * FROM orders TIMESTAMP AS OF date_sub(current_date(), 30) C) SELECT * FROM orders LIMIT HISTORY 30 D) RESTORE TABLE orders TO 30 DAYS AGO

2. After an accidental bulk DELETE removes all rows from a production Delta table, what is the fastest recovery method?
A) Re-run the ingestion pipeline B) RESTORE TABLE orders TO VERSION AS OF C) DROP and recreate the table D) Run VACUUM to recover deleted rows

3. A data engineer writes a DataFrame with an extra column 'discount' not in the existing Delta table schema. The write uses mode='append' and no special options. What happens?
A) Write succeeds; extra column is added B) Write fails — schema enforcement rejects incompatible schema C) 'discount' column is silently dropped D) The table is overwritten

4. Which Delta Lake file compaction operation also supports co-locating related data within files to improve filter query performance?
A) VACUUM B) RESTORE C) OPTIMIZE ZORDER BY D) ANALYZE TABLE

5. A Delta table has been running for 6 months. A data engineer runs VACUUM RETAIN 0 HOURS. What is the consequence?
A) All data is deleted B) All time travel history is permanently removed; only the current version remains queryable C) The table is dropped D) Future writes are blocked

6. What is the primary purpose of the _delta_log/ directory in a Delta table?
A) Store indexes for fast lookups B) Store transaction history as JSON commit files — the source of truth for table state C) Cache query results D) Store column statistics only

7. Which Databricks Runtime type is specifically optimized for SQL and BI workloads using a vectorized C++ query engine?
A) DBR ML B) DBR LTS C) Photon Runtime D) DBR Genomics

8. A team needs a compute option where multiple users run SQL concurrently with fast startup and zero idle billing. What is the best option?
A) All-Purpose Cluster (shared) B) Job Cluster with autoscaling C) Serverless SQL Warehouse D) Single-Node Cluster

9. What happens when you run DROP TABLE on an EXTERNAL table in Unity Catalog?
A) Both metadata and data files are deleted B) Metadata is removed but data files remain at the LOCATION path C) The table is marked deprecated but not deleted D) An error is thrown — external tables cannot be dropped

10. A user is in group A (GRANT SELECT on orders) and group B (DENY SELECT on orders). Can the user query orders?
A) Yes — GRANT takes precedence B) No — DENY always overrides GRANT in Unity Catalog C) Depends on which group was created first D) Yes, if the user also has USE CATALOG and USE SCHEMA

11. Which three privileges are the minimum required for a user to run SELECT * FROM prod.sales.orders?

A) SELECT on orders only B) USE CATALOG on prod, USE SCHEMA on sales, SELECT on orders C) ALL PRIVILEGES on prod D) CREATE TABLE and SELECT on sales

12. A data engineer needs to ensure that analysts can only see rows in a regional sales table where the region matches their own. Which feature implements this?

A) Column masking B) Row filter C) REVOKE on specific rows D) Separate tables per region

13. What is the difference between Liquid Clustering and Z-ORDER BY?

A) They are identical B) Liquid Clustering reorganizes data incrementally via regular OPTIMIZE; columns can change via ALTER TABLE without full rewrite C) Z-ORDER is faster D) Liquid Clustering works only on string columns

14. A team wants a production cluster for nightly ETL that starts fresh, runs in isolation, and is terminated automatically when the job finishes. What cluster type?

A) All-Purpose Cluster B) Job Cluster C) SQL Warehouse D) Single-Node Cluster

15. To reduce Job Cluster startup time from 4 minutes to under 30 seconds without serverless, what should the team configure?

A) Larger instance type B) Instance Pool C) Photon Runtime D) Increase spark.executor.instances

16. Predictive Optimization automates which Delta Lake operations?

A) MERGE and DELETE B) OPTIMIZE and VACUUM C) RESTORE and CLONE D) ANALYZE TABLE and DESCRIBE

17. A table has column 'ssn' (Social Security Number). Analysts should see '*-**-6789' while HR managers see full values. Which feature provides this?**

A) Row filter B) REVOKE SELECT on ssn C) Column masking with a masking function D) Separate tables for each role

18. In the Unity Catalog 3-level namespace, which level is the HIGHEST (top of hierarchy)?

A) Schema B) Table C) Catalog D) Metastore

19. A managed Delta table's data is stored where?

A) In the Databricks control plane B) At a user-specified cloud storage path C) At a location managed by Unity Catalog / Databricks in cloud storage D) In DBFS by default, regardless of Unity Catalog

20. Which Databricks feature automatically captures how data flows from source tables through transformations to dashboards without any manual configuration?

A) Delta transaction log B) Unity Catalog automated lineage C) MLflow tracking D) DESCRIBE HISTORY

Answer Key — Mock Exam

#	Answer	Explanation
1	B	TIMESTAMP AS OF — use date_sub(current_date(),30) to dynamically calculate 30 days ago
2	B	RESTORE TABLE — fastest recovery; creates a new commit pointing to previous version's files
3	B	Schema enforcement fails write — use .option('mergeSchema','true') to allow schema evolution
4	C	OPTIMIZE ZORDER BY — compacts AND co-locates related data within files for data skipping
5	B	Time travel permanently lost — VACUUM RETAIN 0 HOURS removes all non-current Parquet files
6	B	Transaction log — JSON files recording every commit; Delta reconstructs table state from these
7	C	Photon Runtime — C++ vectorized engine; used by SQL Warehouses by default
8	C	Serverless SQL Warehouse — instant startup (<3s), concurrent users, zero idle billing
9	B	Data files remain — external tables only store metadata in UC; DROP only removes the catalog entry
10	B	DENY wins — Unity Catalog DENY is explicit and always overrides any GRANT from any source
11	B	All three required — USE CATALOG + USE SCHEMA + SELECT (each level needs explicit access)
12	B	Row filter — filters entire rows based on user identity; column masking works on column values
13	B	Incremental + flexible — Liquid Clustering integrates with OPTIMIZE and allows ALTER TABLE changes
14	B	Job Cluster — fresh per run, isolated, auto-terminated; perfect for production ETL
15	B	Instance Pool — pre-provisioned VMs reduce startup from minutes to ~30 seconds
16	B	OPTIMIZE and VACUUM — Predictive Optimization automates both based on table usage patterns
17	C	Column masking — apply a masking function that shows full value to hr_managers, masked to others
18	D	Metastore — the top-level container (1 per region); contains all catalogs
19	C	UC-managed location — Databricks/UC manages the storage path; user doesn't specify LOCATION
20	B	Unity Catalog lineage — auto-captured at column level with no configuration required

■ **Tip**

■ **Score Guide:** 18–20: Exam ready! | 15–17: Review managed/external tables & UC privileges | 12–14: Re-study Delta Lake (time travel, VACUUM) and Unity Catalog architecture | Below 12: Return to Week 1–2 content — focus on the cheat sheet concepts

Exam Readiness Checklist

Week 1 & 2 — Self Assessment

E. Week 1–2 Exam Readiness Checklist

■ Topic	Confidence
Lakehouse architecture: lake vs warehouse vs Lakehouse differences	■ Strong ■ Review
Delta Lake ACID properties and how each is implemented	■ Strong ■ Review
Transaction log (_delta_log) structure and purpose	■ Strong ■ Review
Time Travel: VERSION AS OF, TIMESTAMP AS OF, restoreToVersion	■ Strong ■ Review
VACUUM: retention, effect on time travel, DRY RUN	■ Strong ■ Review
MERGE INTO: MATCHED / NOT MATCHED / NOT MATCHED BY SOURCE	■ Strong ■ Review
Schema enforcement (always ON) vs schema evolution (opt-in)	■ Strong ■ Review
OPTIMIZE vs Z-ORDER vs Liquid Clustering differences	■ Strong ■ Review
Predictive Optimization: automates OPTIMIZE + VACUUM	■ Strong ■ Review
Databricks Runtime types: DBR, DBR ML, Photon	■ Strong ■ Review
DBFS vs Unity Catalog Volumes	■ Strong ■ Review
Unity Catalog 3-level namespace: metastore → catalog → schema → table	■ Strong ■ Review
Managed table: DROP deletes data + metadata	■ Strong ■ Review
External table: DROP deletes metadata only; data stays	■ Strong ■ Review
Converting managed ↔ external tables (steps)	■ Strong ■ Review
Unity Catalog lineage: auto-captured, column-level	■ Strong ■ Review
GRANT/REVOKE/DENY syntax and DENY-wins rule	■ Strong ■ Review
Minimum privileges for SELECT: USE CATALOG + USE SCHEMA + SELECT	■ Strong ■ Review
Column masking: masking function + ALTER COLUMN SET MASK	■ Strong ■ Review
Row filter: filter function + ALTER TABLE SET ROW FILTER	■ Strong ■ Review
All-Purpose Cluster: use cases, billing while idle	■ Strong ■ Review
Job Cluster: isolated per run, no idle cost, auto-terminates	■ Strong ■ Review
Instance Pool: reduces startup time to ~30 sec	■ Strong ■ Review
SQL Warehouse types: Classic, Pro, Serverless	■ Strong ■ Review
Serverless: instant startup + zero idle cost	■ Strong ■ Review
Workload-to-compute matching (exam scenario questions)	■ Strong ■ Review

■ **Exam Focus**
■ **Final Strategy for Week 1–2 Topics:**
• **VACUUM** is the most common trap — always check 'what happens to time travel after VACUUM'
• **Managed vs External**: memorize 'DROP managed = data gone; DROP external = data safe'
• **DENY wins**: the most tested Unity Catalog security concept
• **Serverless = fastest startup + cheapest for sporadic workloads**
• **Job Cluster** = always the answer for scheduled production ETL pipelines ■

F. Delta Lake: CLONE & Change Data Feed (CDF)

F1. CLONE — Deep vs Shallow

Clone Type	What Gets Copied	Storage	Use Case
DEEP CLONE	Data files + transaction log (full independent copy)	Full duplicate storage cost	Create isolated copy for testing; data migration
SHALLOW CLONE	Transaction log only; references source data files	Minimal — no data duplication	Cheap test environment; dev/staging without data copy

■ CLONE — Deep and Shallow

```
-- Deep clone: fully independent copy
CREATE TABLE dev_catalog.sales.orders_clone
DEEP CLONE prod_catalog.sales.orders
LOCATION 's3://dev-bucket/delta/orders_clone/';

-- Shallow clone: metadata only (references source files)
CREATE TABLE dev_catalog.sales.orders_shallow
SHALLOW CLONE prod_catalog.sales.orders;

-- Clone at a specific version (for point-in-time copy)
CREATE TABLE backup.orders_jan
DEEP CLONE prod_catalog.sales.orders VERSION AS OF 10;

-- Incrementally sync a deep clone
CREATE OR REPLACE TABLE dev_catalog.sales.orders_clone
DEEP CLONE prod_catalog.sales.orders;
```

F2. Change Data Feed (CDF)

Change Data Feed tracks row-level changes (INSERT, UPDATE, DELETE) made to a Delta table. Downstream pipelines can read only changed rows instead of full table scans — enabling efficient incremental processing.

■ Change Data Feed

```
-- Enable CDF on a table
ALTER TABLE prod_catalog.sales.orders
SET TBLPROPERTIES (delta.enableChangeDataFeed = true)

-- Or at creation time
CREATE TABLE orders (...) TBLPROPERTIES (delta.enableChangeDataFeed = true)

-- Read changes between versions
changes = spark.read.format('delta') \
    .option('readChangeFeed', 'true') \
    .option('startingVersion', 5) \
    .option('endingVersion', 10) \
    .table('prod_catalog.sales.orders')

# CDF adds 3 metadata columns:
# _change_type:    'insert','update_preimage','update_postimage','delete'
# _commit_version: Delta version number
# _commit_timestamp: Timestamp of commit

changes.filter("_change_type = 'insert'").show()
```

■ **Tip**

CDF Use Cases: Incremental ML feature updates · CDC pipelines · Audit trails · Silver layer incremental processing from Bronze. Note: CDF adds storage overhead — only enable when needed.

G. Databricks Workspace: Collaboration & File Management

Feature	Description	Key Points for Exam
Notebooks	Interactive Python/SQL/Scala/R; real-time co-authoring	Each cell has language magic: %python, %sql, %scala, %r, %sh, %md
Databricks Git Folders	Connect notebooks to GitHub/GitLab/Azure DevOps/Bitbucket	Create branches, commit, push, PR all from within Databricks UI
Workspace Files	Store arbitrary files (Python modules, configs) in workspace	Import Python modules with %run or sys.path; not version-controlled like Git Folders
Shared Objects	Notebooks, experiments, models shared via workspace permissions	Set Can View / Can Run / Can Edit / Can Manage at notebook or folder level
Clusters in Notebooks	Attach a notebook to a running cluster	Multiple notebooks can share one all-purpose cluster; each has isolated session state
Widgets	Interactive parameters in notebooks (text, dropdown, multiselect, combobox)	dbutils.widgets.text("param","default"); dbutils.widgets.get("param")

■ Notebook Magic Commands & dbutils

```
# ■■■ Notebook Magic Commands ■■■
%python   # switch cell to Python
%sql      # switch cell to SQL
%scala    # switch cell to Scala
%r        # switch cell to R
%sh       # run shell command
%md       # Markdown text cell
%run /path/to/other/notebook  # run another notebook inline
%fs ls /delta/orders/         # browse DBFS

# ■■■ dbutils – Databricks Utilities ■■■
dbutils.fs.ls('/delta/')              # list files
dbutils.fs.cp('/src/', '/dst/')       # copy
dbutils.fs.rm('/path/', recurse=True) # delete

dbutils.secrets.get('scope','key')    # retrieve secret

dbutils.widgets.text('start_date', '2026-01-01')
start = dbutils.widgets.get('start_date')

dbutils.notebook.run('/path/child_nb', timeout_seconds=300,
    arguments={'param': 'value'})     # run & wait for result

dbutils.notebook.exit('success')      # return value from notebook
```

H. Common Exam Traps & Edge Cases

Trap	Wrong Answer	Correct Understanding
VACUUM removes ALL history	VACUUM deletes the entire table	VACUUM only removes data files OUTSIDE the retention window. Current data is safe. Only time travel beyond retention is lost.
Schema evolution is automatic	Delta always adds new columns automatically	Schema ENFORCEMENT is always on. Schema EVOLUTION (new columns) requires .option("mergeSchema","true") — it is opt-in.
DROP TABLE on external = data deleted	All table drops delete data	EXTERNAL tables: DROP only removes UC metadata. Data files at LOCATION path are untouched.
USE CATALOG is enough to query	GRANT USE CATALOG grants SELECT access	USE CATALOG only lets you navigate to the catalog. You still need USE SCHEMA + SELECT on the specific table.
Serverless = most expensive	Serverless costs more due to managed infrastructure	Serverless has ZERO idle cost. For sporadic workloads it is cheaper than classic clusters that run idle.
Job Cluster = shared like All-Purpose	Job Cluster can be used by multiple jobs simultaneously	Job Cluster is created fresh per job run, completely isolated. It terminates when the job finishes.
DENY is just another permission	DENY and REVOKE are similar — REVOKE is stronger	DENY explicitly blocks access and OVERRIDES any GRANT, including grants from group membership.
Managed vs External = same on DROP	Both types retain data after DROP TABLE	Managed = data deleted on DROP. External = data stays. This distinction is tested constantly.
Z-ORDER can change columns freely	Z-ORDER can be updated without full rewrite	Changing Z-ORDER columns requires a new full OPTIMIZE run (expensive). Liquid Clustering allows column changes via ALTER TABLE without full rewrite.
Photon speeds up all workloads equally	Photon accelerates Python UDFs and ML training	Photon primarily accelerates SQL and DataFrame workloads. Python UDFs and MLlib training do not benefit significantly from Photon.

■ Exam Focus
Study these traps carefully — each one represents a common wrong answer pattern on the exam. The exam frequently tests the exact boundaries between similar-sounding features.

I. Master Reference: Delta Lake SQL Commands

Command	Syntax	Key Options / Notes
CREATE TABLE	CREATE TABLE cat.sch.tbl (cols) USING DELTA [PARTITIONED BY] [LOCATION] [TBLPROPERTIES]	No LOCATION = Managed; LOCATION = External
CREATE TABLE AS	CREATE TABLE tbl AS SELECT ... (CTAS)	Auto-infers schema; must be Delta in UC
INSERT INTO	INSERT INTO tbl [VALUES (...) | SELECT ...]	Appends; use INSERT OVERWRITE to replace
INSERT OVERWRITE	INSERT OVERWRITE tbl SELECT ...	Replaces all data atomically (version++)
UPDATE	UPDATE tbl SET col=val WHERE condition	Full ACID; creates new version
DELETE	DELETE FROM tbl WHERE condition	Full ACID; tombstones files
MERGE INTO	MERGE INTO target USING source ON condition WHEN MATCHED ... WHEN NOT MATCHED ...	Most powerful DML; handles upsert + delete
OPTIMIZE	OPTIMIZE tbl [WHERE partition_filter] [ZORDER BY (cols)]	Compacts small files; improves read performance
VACUUM	VACUUM tbl [RETAIN n HOURS] [DRY RUN]	Removes files outside retention; irreversible
RESTORE	RESTORE TABLE tbl TO VERSION AS OF n / TIMESTAMP AS OF ts	Creates new version pointing to old state
DESCRIBE HISTORY	DESCRIBE HISTORY tbl [LIMIT n]	Shows all commits with operation, user, timestamp
DESCRIBE DETAIL	DESCRIBE DETAIL tbl	Shows numFiles, sizeInBytes, partitionColumns, etc.
ALTER TABLE	ALTER TABLE tbl ADD/DROP/RENAME COLUMN; SET TBLPROPERTIES; CLUSTER BY; SET MASK	Schema changes; property updates; clustering
ANALYZE TABLE	ANALYZE TABLE tbl COMPUTE STATISTICS [FOR COLUMNS ...]	Refreshes column statistics for data skipping
CLONE	CREATE TABLE new DEEP|SHALLOW CLONE src [VERSION AS OF n]	Deep = full copy; Shallow = metadata only

■ Delta Lake — Essential Patterns

```
-- Quick reference: most common Delta patterns

-- 1. Upsert (most tested)
MERGE INTO target t USING source s ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *

-- 2. Safe time travel query (check retention first!)
DESCRIBE HISTORY orders LIMIT 5;  -- check oldest version
SELECT * FROM orders VERSION AS OF 3;

-- 3. Incremental optimization
OPTIMIZE orders WHERE order_date >= current_date() - 7
ZORDER BY (customer_id, order_date)
```

```
-- 4. Safe VACUUM with dry run
VACUUM orders RETAIN 168 HOURS DRY RUN;  -- preview
VACUUM orders RETAIN 168 HOURS;           -- execute
```